

## funzioni di costruzione sistemi lineari (da migrare in libreria)

### `Vec derivata_discreta_come_matrice(const Mat &D, const Vec &u)`

Applica l'operatore derivata discreta rappresentato dalla matrice delle differenze finite in avanti  $D$ , restituendo  $Du$ .

- **Parametri:**
  - $D$ : matrice delle differenze finite di dimensione  $(n - 1) \times n$ .
  - $u$ : segnale campionato su griglia uniforme.
- **Note implementative:**
  - Solo un wrapper a `matrix_prodotto_vector(D, u)`, utile come punto di separazione concettuale tra "derivata come matrice" e "derivata come operazione implicita".
  - La matrice  $D$  è prodotta da `matrix_build_derivata1(...)`, che già espone gli elementi  $-1/h$  e  $1/h$ .
- **Stato:** da migrare in libreria.

### `Vec derivata_discreta_come_RI(const Vec &u, const double a, const double b)`

Calcola la derivata discreta senza costruire la matrice  $D$ , usando direttamente la formula delle differenze in avanti su intervalli noti.

- **Parametri:**
  - $u$ : segnale campionato di dimensione  $n$ .
  - $a, b$ : estremi dell'intervallo di discretizzazione.
- **Note implementative:**
  - Restituisce un vettore di dimensione  $n - 1$  con  $(u[j + 1] - u[j])/h$ , dove  $h = (b - a)/(n - 1)$ .
  - Il vantaggio didattico è mostrare che l'operatore lineare non richiede esplicitamente la sua matrice per essere valutato.
  - Il sorgente contiene un TODO esplicativo in commento: istruire lo studente a derivare la formula del ciclo da solo.
- **Stato:** da migrare in libreria.

### `void costruisci_sistema_cauchy(int n, double t0, double T, double x0, Vec &t, Vec &fvals, Mat &L, Vec &b, double (*ft)(double), bool backw)`

Costruisce il sistema lineare  $Lx = b$  per il problema di Cauchy  $x'(t) = x(t) + f(t)$ ,  $x(t_0) = x_0$ , discretizzato con differenze finite in avanti o all'indietro.

- **Parametri:**
  - $n$ : numero di nodi della discretizzazione.
  - $t_0, T$ : estremi dell'intervallo temporale.
  - $x_0$ : condizione iniziale.
  - $t$ : vettore in uscita con i nodi temporali.
  - $fvals$ : vettore in uscita con il forcing  $f(t_i)$  campionato.
  - $L$ : matrice del sistema  $n \times n$ .

- **b**: termine noto.
- **ft**: callback per la funzione forzata.
- **backw**: se **true**, usa lo schema alle differenze all'indietro; se **false**, schema in avanti.
- **Note implementative:**
  - Schema in avanti:  $L[i][i] = 1, L[i][i-1] = -(1+h), b[i] = h \cdot f(t_{i-1})$ , con  $L[0][0] = 1$  come condizione iniziale.
  - Schema all'indietro:  $L[i][i+1] = 1, L[i][i] = -(1+h)$ , con  $b[n-1] = x_0$ .
  - La matrice  $L$  è tridiagonale sparsa, ma viene conservata in forma densa.
- **Stato**: da migrare in libreria — valutare se esporre **t**, **fvals** come output separati oppure incapsularli in una struct di risultato.

```
void costruisci_sistema_bordo(int n, double t0, double T, double x0,
double x1, Vec &t, Vec &fvals, Mat &A, Vec &b)
```

Costruisce il sistema lineare  $Ax = b$  per il problema al contorno  $x''(t) = f(t)$  con condizioni di Dirichlet  $x(0) = 0, x(1) = 0$ , usando differenze finite centrali.

- **Parametri:**
  - **n**: numero di nodi della discretizzazione su  $[0, 1]$ .
  - **t0, T**: estremi dell'intervallo.
  - **x0, x1**: valori al contorno.
  - **t**: vettore in uscita con i nodi.
  - **fvals**: forcing campionato.
  - **A**: matrice tridiagonale  $n \times n$ .
  - **b**: termine noto.
- **Note implementative:**
  - $A[i][i] = -2/h^2, A[i][i-1] = A[i][i+1] = 1/h^2$ .
  - Matrice tridiagonale simmetrica e definita negativa, ben condizionata per griglie uniformi.
  - Il problema è utile per introdurre ill-conditioning in spazi discreti più fini, vedi **f2\_norme** nel main.
- **Stato**: da migrare in libreria.